长程工作流：Harness、Loop 与 Graph

张虎来

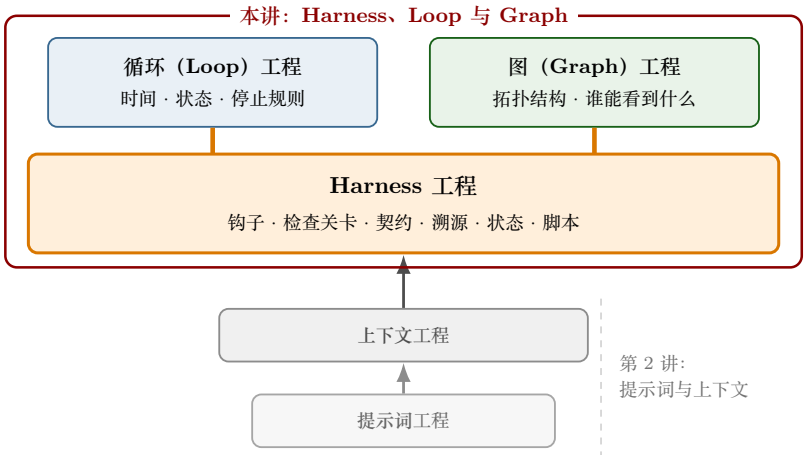
复旦大学国际金融学院

2026 年秋季

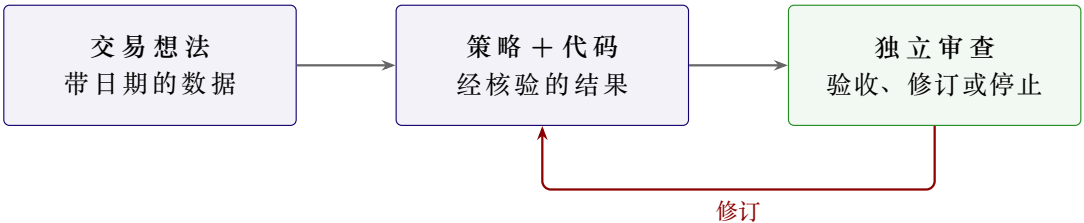


工程层次

Loop 与 Graph 建立在 Harness 之上，而非更高一层的阶梯。

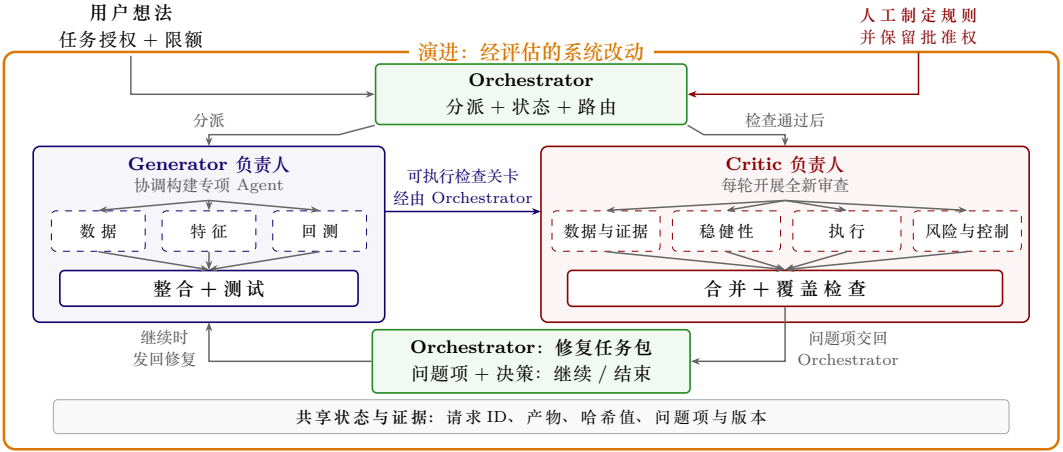


构建智能体量化系统



- 把交易想法与带日期的数据转化为有证据支撑的策略。
- 返回证据与局限；验收需要可追溯、可复算、可恢复的工作。

完整架构

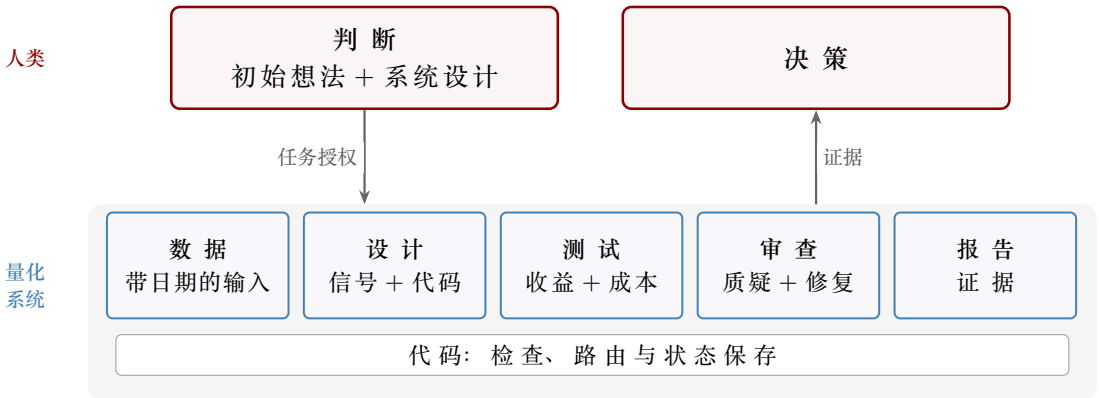


- 演进在各次运行之间评估系统改动；最终批准权仍由人掌握。

金融：主要差异

- **重大后果**
  - 错误可能影响客户、资金和机构义务，部分操作难以撤销。
  - 支持结论的证据本身不构成行动授权。
- **时间信息**
  - 金融结果取决于决策时点可得的信息与产生该结果的版本。
  - 应区分重建已发生的运行、复算固定计算和新的模型运行。
- **边界授权**
  - 需要明确谁能行动、授权范围、限额以及谁能停止工作流。
  - 研究验收、版本批准和资金使用是不同决定。
- **持续运行与受控变更**
  - 中断后恢复已核验工作；评估改动时保留既有控制与旧版本。

职责分工



- 人选择初始想法与系统设计，再决定如何使用资金。

提纲

- **长程工作流设计**
  - Harness：运行环境与控制
  - Loop：循环与停止
  - Graph：依赖与交接
  - Evolution：经评估的系统改动
- **量化系统案例**
  - 构建 QT 组件与检查
  - 演进一个可复用的系统版本

Orchestrator、Generator 与 Critic

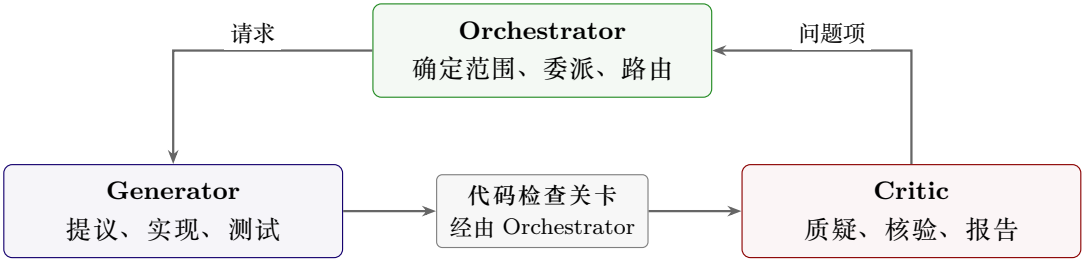
Harness
运行环境与控制

循环
重复执行与停止

图
依赖关系与任务交接

演进
经过评估的系统改动

为何分开角色?



Orchestrator 发出修复请求，或记录终止结果。

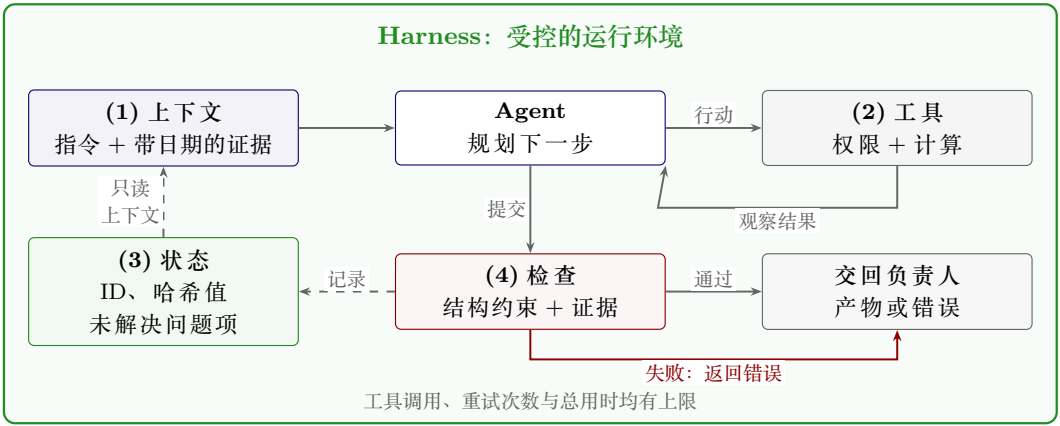
- 主会话协调工作；控制器代码检查每一次状态转换。
- Generator 和 Critic 都不能自行扩大权限或绕过检查。

角色契约

设计问题	职责分离
产出方自行验收	Critic 在独立上下文中检查证据
审查中改变目标	任务授权与验收标准保持不变
控制权归属不清	Orchestrator 决定路由；代码核验状态转换

- 独立的上下文有助于 Critic 质疑方案，但两个 Agent 仍可能犯同样的错误。
- 在任务委派和独立核验确有价值时采用这些角色。

每个 Harness 内部

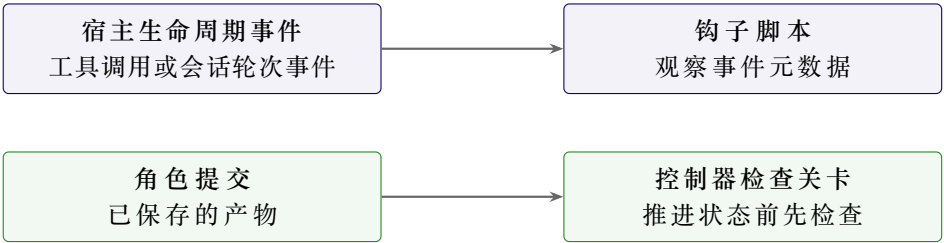


- 四个编号组件共同构成每个角色的 Harness；控制器拥有运行状态并负责推进流程。

契约与边界

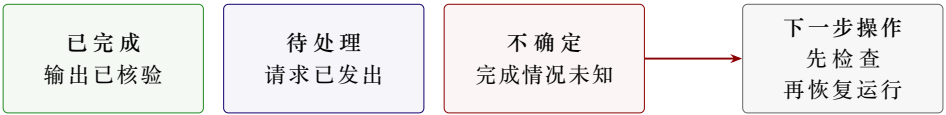
- **任务与产物契约**
  - 明确任务、固定输入、权限、返回格式及完成/失败条件。
  - 每份产物都关联请求及其支持证据。
- **权限边界**
  - 披露文件和网页是数据，不是权限来源。
  - 付费数据请求仍需核验用途、供应商、金额、有效期、身份和批准人。
  - 课堂 QT 没有支付或券商权限。

Hook 与状态控制器



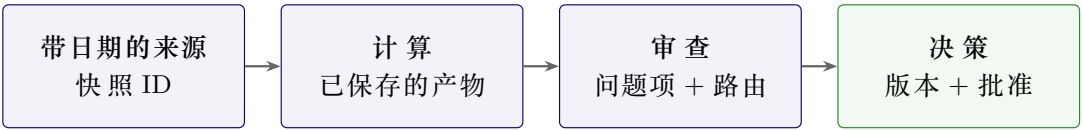
- **Hook**
  - 宿主事件发生时运行回调，记录事件元数据。
  - 只有宿主支持事件阻断时，才可能阻止操作。
- **状态控制器**
  - 核验提交产物、哈希、问题项和数值检查。
  - 决定运行是前进、重试、等待还是停止。
- Hook 观察事件；控制器管理工作流状态。

状态与检查点



- (1) 身份：案例、请求、候选方案和提示词/版本 ID。
- (2) 进度：待处理角色、轮次、重试次数和截止时间。
- (3) 证据：用于发现产物变化的路径与哈希。
- (4) 问题项：未解决问题和已核验修复。
- 每次状态转换时保存已核验状态；对话最后一行不能说明哪些步骤已经完成。

审计证据

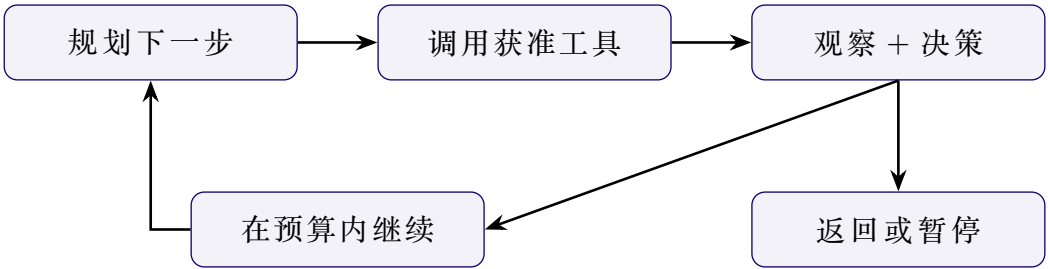


- 将 JSON 证据保存在本地，并保留 Git 版本历史。每次批准都记录批准人、时间戳，以及获准的操作或版本。
- 另一位分析师可以据此重建可观察的操作与文件；记录并不揭示模型隐藏的推理过程。

循环提纲

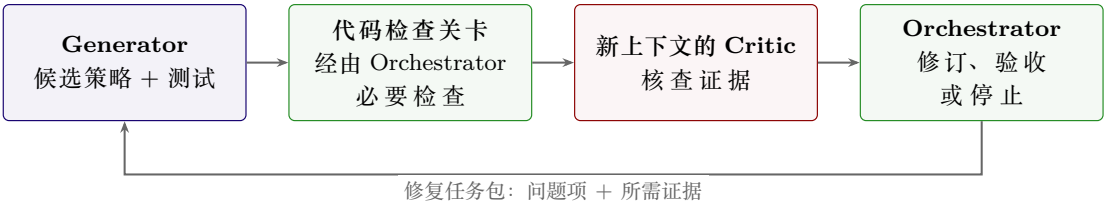
- 循环
  - 行动、观察、决策
  - 修订与审查
  - 携带问题项并核验修复
  - 验收、停止与恢复

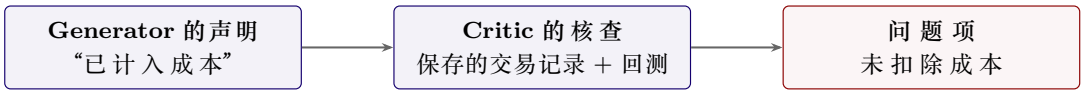
行动、观察与决策



- 工具调用失败后，任务仍未完成。
- 根据观察结果选择下一步行动。
- 对暂时性错误进行有限次数的重试；无效证据不得推动案例进入下一步。

修订循环





- 向 Critic 提供已保存的交易记录和回测输出，供其核查。
- 每轮启用新上下文的 Critic 可减轻对先前回答的锚定；但不同 Agent 仍可能犯同样的错误。
- 修订过程中保持任务授权与验收规则不变。

问题项与修复

时序问题

信号使用第 t 日的收盘价。

回测却在同一收盘时点交易。

交易必须发生在信号可用之后。

问题项台账

第 1 轮：记录 `pit-001`，并链接至受影响的输出。

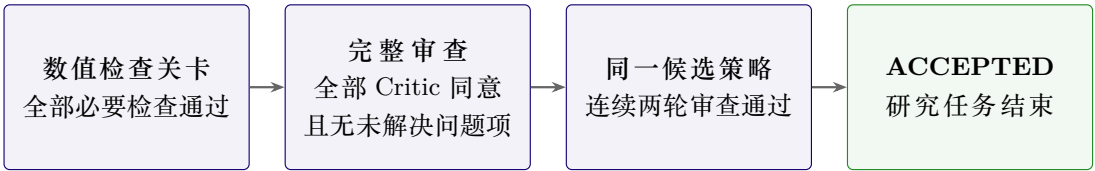
第 2 轮遗漏该项：问题项仍保持未解决。

只有 Critic 核验修复后的输出，才能关闭该问题项。

- 问题：对已保存文件做什么检查，才能证明时序问题已经修复？
- 下一次请求须包含所有尚未解决的问题项。

举手回答

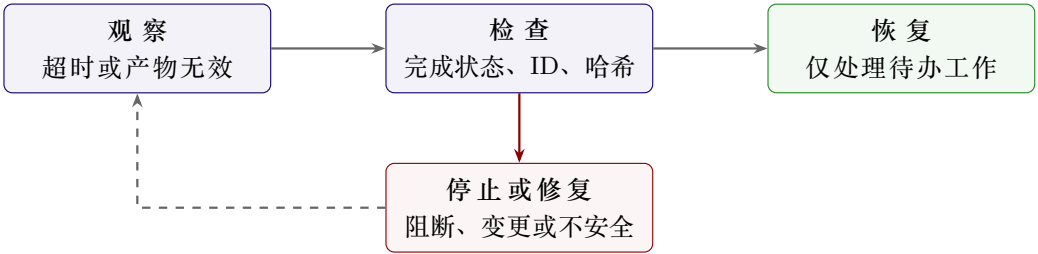
验收



候选策略发生变化后，连续通过审查的轮数重新计算。

- QT 要求同一候选策略连续两轮通过全部审查；更换候选策略后，连续通过轮数重新计算。
- 这套课堂验收规则既不能证明策略盈利，也不构成交易授权。

停止与恢复

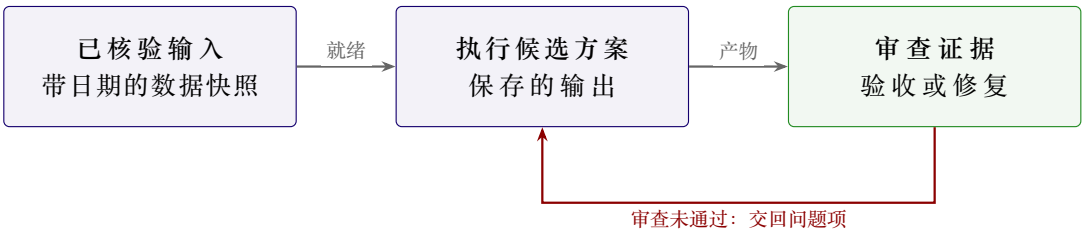


- 开始前设定轮数、尝试次数、工具调用次数和用时上限。
- 继续是非终止路由；否决、阻断、耗尽、停滞和用户停止是不同结果。
- 重试前核验完成状态、修复错误、调查证据变化并核对不确定的外部操作。
- 只恢复已核验的待办工作；已完成案例直接返回保存的结果。

图提纲

- 图
  - 依赖与交接
  - 产物与共享状态
  - 团队、分发与汇合
  - 条件路由与合并限额

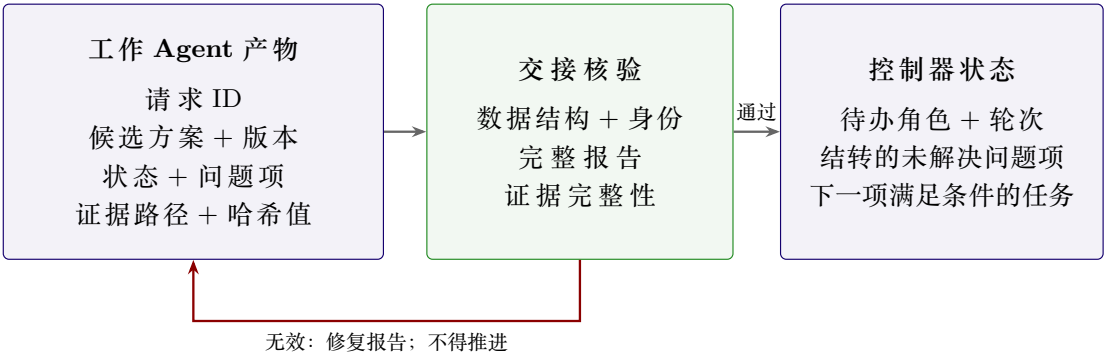
依赖与交接



前置条件未满足：先等待，再分派任务。

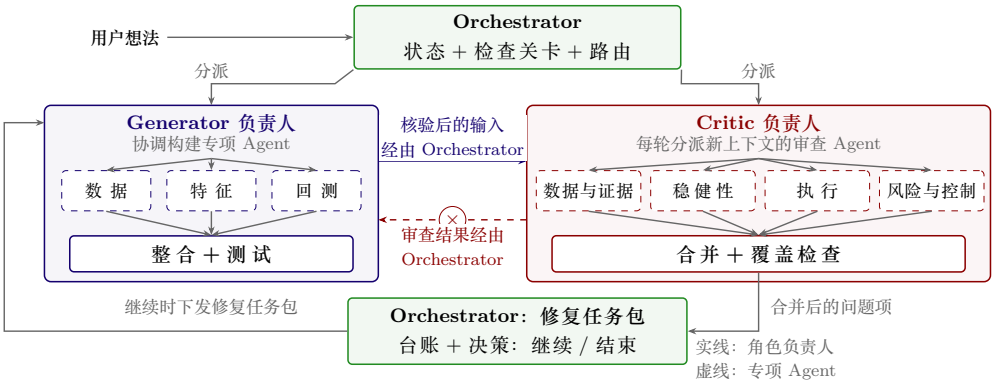
- 节点执行任务；边规定前置条件与交接方式。
- 缺少输入时先等待；审查未通过时，将结果交回 Generator。
- 前向任务链可以无环；加入修复边后，完整工作流便包含循环。

产物与共享状态



- 各工作 Agent 分别编写报告。只有控制器可以更新运行的控制状态。
- 合并前核验身份、完整性与证据。
- 明确保留相互冲突的问题项；最后提交的报告不得悄然将其覆盖。

团队与汇合



- 检查需要不同专业知识时，使用专项 Agent。
- 等待所有必需报告提交，再核验身份与证据，之后才能合并结果。

条件路由



输入版本变化时启动新案例，并重建依赖产物。

- 依据已核验状态与依赖关系选择下一项任务。
- 输入缺失时等待；提交无效时在同一角色修复；检查通过前保持下游工作阻断。
- 输入版本变化时启动新案例并重建依赖产物。
- 局部检查通过不保证合并后的敞口有效；课堂样例没有组合层面的检查关卡。

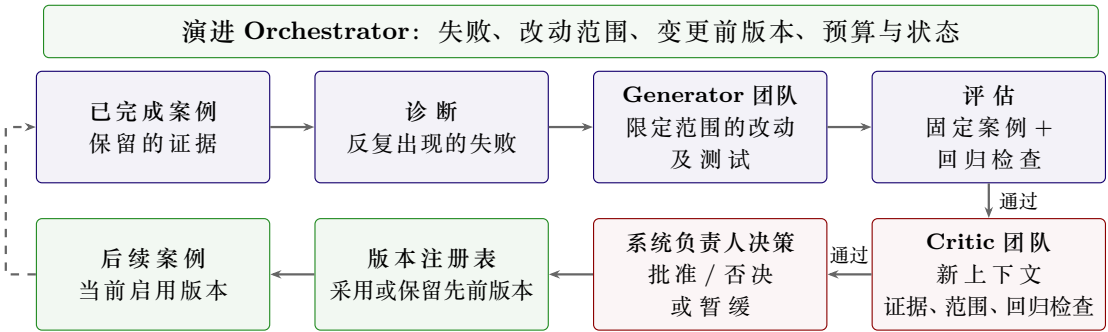
演进提纲

- 演进
  - 可改变的内容：提示词/上下文、技能、工具/代码、角色/工作流
  - 证据与评估
  - 回归审查
  - 版本控制、批准与监测

演进

- 工具循环选择行动；研究循环修订候选策略。
- 演进评估可复用系统改动，并决定采用、否决或暂缓。
- 策略参数搜索仍属于研究；QT 通过提示词版本演示演进。

演进工作流



- Generator 提出限定范围的改动与测试；新上下文中的 Critic 团队审查证据、回归检查与改动范围。
- 只有通过必要检查与审查的版本才可获批。改动若被否决或暂缓，继续保留当前启用版本。

证据与评估

- 比较改动前先确定标准。
- 检查格式、含义、证据支持与授权。
- 使用开发案例获取反馈，使用留出的案例进行最终评估。
- QT 时序检查要求在信号实际可用之后执行；仅有日期字段不充分。

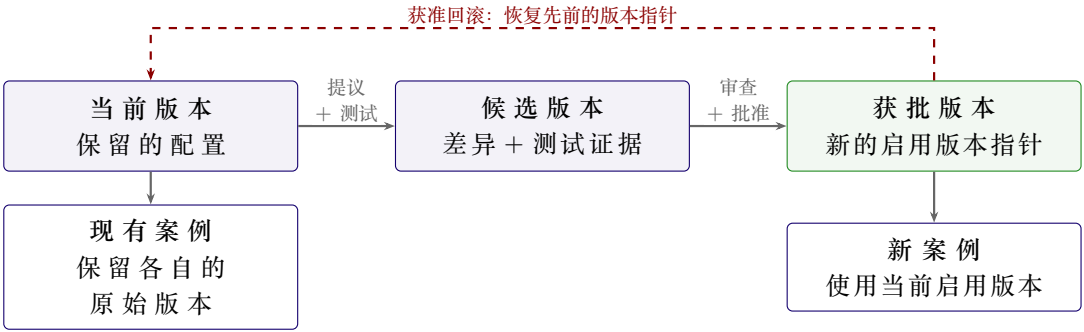
开发案例
反馈可用于修订

预留案例
用于单独的最终评估

回归审查

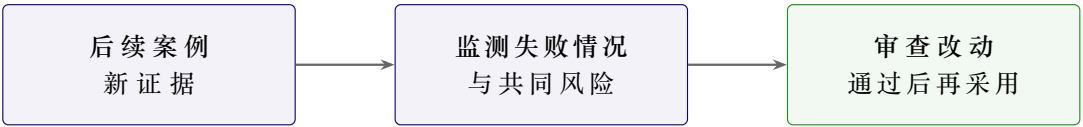
- 控制器测试检查缺失报告、过期 ID、无效证据和权限。
- 模型行为比较检查时序、证据以及是否修复记录的失败。
- QT 提示词接口不接受风险限额、策略参数和控制器代码改动。
- 改动提案不能降低自身验收标准。

版本与回滚



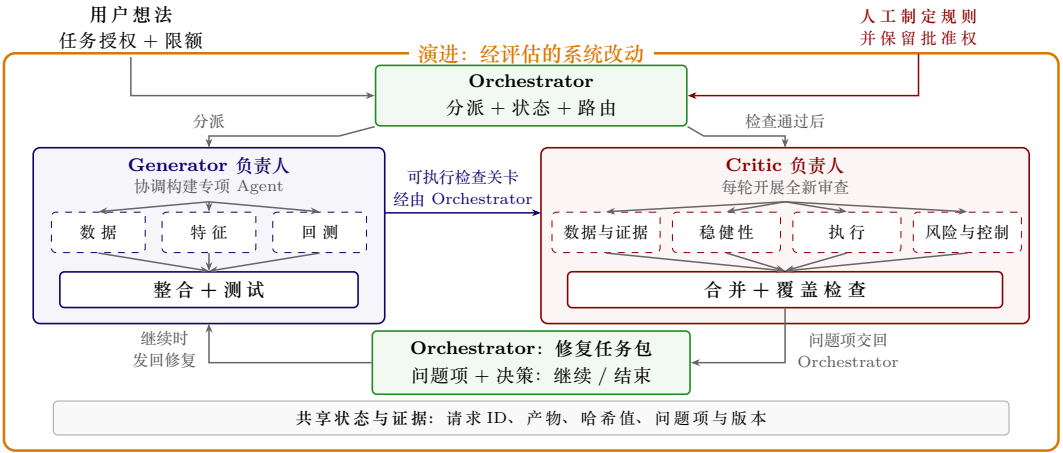
- 为每个版本记录测试时使用的模型、工具、数据、规则和提示词。
- 别名用于选择版本；批准仍需独立记录。
- 保留新旧两个版本及其证据。回滚会恢复先前的版本指针，并须在新案例中核验行为。

监测



- 监测反复出现的失败与共同风险。
- 模型、工具、数据源或权限变化时，重新评估。
- 将已记录的失败反馈给演进流程。提示词优化并不保证可靠性。

完整架构



- 定位角色 Harness、修订循环、交接路径与演进边界。

提纲

- 构建：插件、角色、检查关卡、案例运行与生产边界。
- 演进：记录失败、比较、批准、回滚与监测。

插件结构与入口

```
qt/
  kimi.plugin.json         # commands, skills, agents
  commands/{start,evo}.md  # two public entry points
  skills/{start,evo}/      # Orchestrator workflows
  agents/                  # two leads + six specialists
  references/              # schemas, role cards, policies
  runtime/quant_cn/        # state machine and numerical gates
  scripts/{qt,check}.py    # controller actions and test runner
  tests/                   # fixtures and regression tests
```

- 用 /qt:start 开展研究，用 /qt:evo 改进系统。
- 用 /plugins install <qt> 安装。修改源码后，重新安装并运行 /reload。
- 将安装后的代码与工作区 .qt/ 中的案例状态、提示词版本分开保存。

负责人与专项 Agent

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- 在角色卡中定义负责人的工具权限及允许调用的专项 Agent。
- 实际运行时，检查宿主的实际分派与响应记录。
- 仅凭报告文件，无法证明各专项 Agent 曾在独立上下文中运行。

Orchestrator 与检查关卡

- Orchestrator 绑定请求，核验 Generator 报告与数值关卡，分派新 Critic 审查并拥有状态转换。
- 前进前必须有三份匹配的 Generator 报告和三份新的 Critic 审查。
- 运行 `python3 <qt>/scripts/check.py` 检查控制器规则。

```
reports, paths = load_team(output, response, request)
if state["role"] == "generator":
    generator_gate(output, state, response, round_dir)
else:
    merge_critic(state, reports, request)  # validate without mutating state
```

Hook

```
{"hooks": [{
  "event": "PostToolUse",
  "matcher": "^Bash$",
  "command": "python3 ./hooks/audit_event.py",
  "timeout": 3
}]}
```

- 可选 Hook 练习（未随插件提供）：将 `audit_event.py` 复制到 `<qt>/hooks/`，合并清单配置，再重新安装并加载。
- 事件名、工具名及会话与调用 ID 只记录一次；它既不评估候选策略，也不提供完整的专项 Agent 执行轨迹。
- Kimi 记录 Hook 错误或超时后会继续运行。必要检查仍由控制器执行。

运行一个研究案例

- 跟踪 `request_id`、报告、数值输出、路由、`trace.jsonl` 和终止结果。
- 按决策日期选择当时可用的时点版本；重述版本在披露前不可用。
- 动量契约使用 21 个有效收盘价：$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$，示例为 $12/10 - 1 = 20\%$。
- 在案例前检查稳定排序与最早可执行时间。
- 样例只测试动量候选方案；仅 `lookback` 和 `top_n` 变化。
- 脚本演练需明确标注；声称真实团队运行时必须出示宿主分派记录。

修复被退回的提交

- 缺少专项报告：否决提交，留在当前角色并消耗一次尝试。
- 请求 ID 过期：执行同样的否决与重试规则；第二次无效提交变为 `BLOCKED`。
- 报告完整匹配后，先检查证据再前进。
- 问题：在临时测试案例中制造缺少报告和过期 ID，哪些修复才能继续?
- 不带 `--response` 调用 `advance`；重新读取 `request.json`，让所有报告匹配新 ID。检查哈希并保留封存证据。

举手回答

重放、审计与恢复

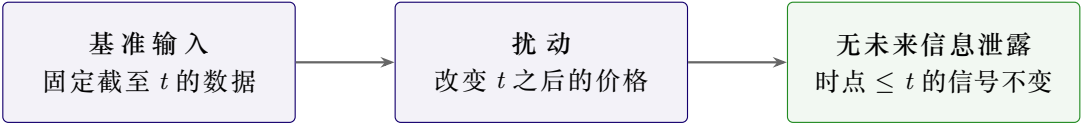
- 重建：检查输入/代码、已解决与未解决问题项、停止规则、角色卡/版本以及批准记录。
- 复算：固定输入、代码和配置，比较数值结果。
- 新的模型运行：使用新案例，保留自己的决定、工具路径和输出。
- 恢复或复用：核验请求 ID 与哈希；恢复待办工作；已完成案例返回保存结果。
- 问题：重建本案例。案例文件或单独批准记录是否标明了批准该版本的人?

举手回答

生产数据

- 输入：有许可证、带日期的市场与财务数据。
- 股票池：历史成分、停牌与退市。
- 执行：相关板块和日期的手数、价格限制、T+1 规则与费用。
- 运营：宿主隔离、责任人，以及独立测试的适配器。
- 课堂样例使用五只虚构股票和合成工作日数据；生产数据接入、公司行为、组合控制和券商适配器仍待构建。

研究验证



- 检验未来价格变动、数据重述、更高成本，以及依次推进的样本外区间表现。
- 通过这些检查，并不能证明策略将获得超额收益。

资金授权



- 检查组合限额、当前持仓、批准有效期与实际成交。
- 由指定负责人批准资金使用；研究验收通过不等于获得下单授权。
- `trade_sheet.csv` 使用独立的历史合成仓位计算样例，并非来自已验收的候选策略。QT 没有券商连接。

记录一次失败

- 回到研究工作流，/qt:evo 开始系统改进；evo-prepare 关联完成案例与变更前提示词版本。
- 记录失败、所需行为、开发案例和独立的最终评估案例。
- GEPA 等外部候选工具可以提出改动，但未集成到 QT。

```
python3 <qt>/scripts/qt.py evo-prepare --case case-01 --change change-01
```

提示词改动

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

- 问题：哪条指令针对了时序遗漏？需要什么证据才能说明它有效？
- 该 QT 接口只修改 Generator 提示词。策略参数、风险限额、验收规则与控制器代码均不在其修改范围内。

举手回答

比较输出

字段	基准版本	候选版本
信号可得时间	29 Sep 15:00	29 Sep 15:00
执行时间	29 Sep 15:00	30 Sep 09:30
证据 ID	`PRICE_01`	`MISSING_02`

- 交易从 30 Sep 09:30 (UTC+8) 起才允许执行。仅留存了 `PRICE_01`。
- 运行配套检查，或在 promptfoo 中加载 `promptfooconfig.json`，比较两份输出。
- 问题：是否有任一输出同时通过两项检查?

举手回答

判断证据

检查项	基准版本	候选版本
执行时序	未通过	通过
证据引用	通过	未通过
采用决定	否决	否决

- 候选版本修正了时序，却引用了缺失的来源 `MISSING_02`。
- 问题：采用该版本前，还需修复哪些问题，并进行哪些独立检查？

举手回答

版本控制

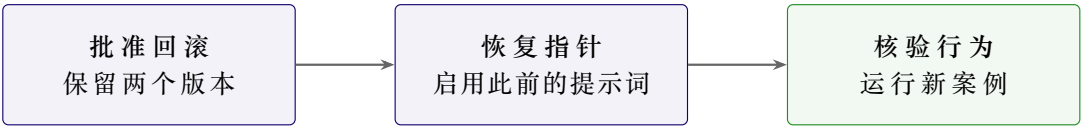
- Git 记录源代码历史以及产生候选改动的差异。
- QT 注册表记录提示词字节、活动指针、父版本和运行配置。
- MLflow 是外部对应工具，用于提示词版本、评估与追踪；未集成到 QT。
- 别名用于选择版本，但批准需要独立记录。

评估、测试、批准

- (1) 评估：使用提案、审查和回归结果运行 `evo-evaluate`。
- (2) 测试：检查代码回归和行为比较，包括留出的案例。
- (3) 批准：只有明确的人类批准才能运行 `evo-adopt`；否决则保留活动版本。
- 研究验收、版本采用和资金授权仍是不同决定。

举手回答

回滚与监测



- 获得批准后，用 `evo-rollback` 恢复此前的指针。保留新旧两个版本及其证据。
- 已有案例保留原配置，运行中途不切换版本。
- 在新案例中核验恢复后的版本，再监测后续运行中是否反复出现失败。

最终审查

- 证据：追溯并复算已记录工作。
- 控制：要求失败时阻断或解释。
- 连续性：恢复已核验工作，避免重复不确定操作。
- 改进：评估并恢复受治理的系统版本。
- 选择一项控制，说明它回应谁的需要，并说明还需构建什么。

举手回答